

UNIVERSIDADE FEDERAL DE GOIÁS

Instituto de Informática

Bacharelado em Engenharia de Software

Disciplina: Software Concorrente e Distribuído — 2026.1

Professor: Fábio Moreira Costa

SPACESHIP

Plataforma de Jogos de Naves Multiplayer em Tempo Real

Documentação Técnica do Trabalho Final

Projeto e Implementação de um Sistema de Software Concorrente e Distribuído

Cenário escolhido: Exemplo 3 — Jogo online multijogador

Equipe

Moisés Ferreira Protázio Bastos — Game Design

Thiago Nascente Borges — Programação

Vinicius Silva Benevides — Programação

Ana Liz Bomfim Gomes — Testes / QA

Goiânia — 2026

Repositório do Código-Fonte

Esta é uma entrega independente. O código-fonte completo do projeto (todos os serviços, scripts de deploy e dados de teste) está no repositório indicado abaixo. Utilize o espaço reservado para registrar o link oficial de entrega.



Repositório (GitHub)

<https://github.com/DistributedSystems-UFG/trabalho-final-grupo-6-1.git>

Sumário

Os números de página abaixo já vêm preenchidos. Caso o documento seja editado, é possível atualizar o sumário no Word/LibreOffice clicando sobre ele e pressionando F9.

1. Visão Geral do Projeto	5
1.1. Objetivo	5
1.2. Os três minijogos	5
1.3. Características-chave	6
2. A Proposta do Jogo (Game Design)	7
2.1. Tema e mecânicas base	7
2.2. Controles e regras universais	7
2.3. Fluxo do jogador	7
2.4. Minijogo 1 — Duelo (PvP 1v1)	8
2.5. Minijogo 2 — Chuva de Asteroides (Survival)	8
2.6. Minijogo 3 — Invasão Alienígena (Cooperativo + Boss)	8
2.7. Inimigos (Minijogo 3)	9
2.8. Interface do usuário (telas)	9
2.9. Identidade visual	10
2.10. Áudio	10
3. Stack Tecnológica	11
3.1. Por que mais de uma linguagem	11
4. Arquitetura Distribuída	13
4.1. Estilo arquitetural	13
4.2. Serviços e responsabilidades	13
4.3. Diagrama da arquitetura	14
4.4. Paradigmas de comunicação	16
4.5. Legenda dos fluxos do diagrama	16
4.6. Concorrência em cada serviço	17
4.7. Replicação e particionamento	17
4.8. Consistência e disponibilidade	17
5. Funcionamento do Sistema (fluxo de ponta a ponta)	19

5.1. Autenticação e entrada	19
5.2. Lobby e partida em tempo real	19
5.3. Fim de partida e ranking (pipeline assíncrono)	19
6. Estrutura de Pastas	21
7. Detalhes da Implementação	23
7.1. Frontend (cliente web)	23
7.2. Auth Service (C# / ASP.NET Core)	23
7.3. Auth Service Lite (Node.js)	23
7.4. Game Gateway (relay)	23
7.5. Game Engines (3 processos isolados)	24
7.6. Score Service (Python / Flask)	24
7.7. Produtor Kafka (resiliência)	25
7.8. Persistência: particionamento e replicação	25
7.9. Configuração (nada chumbado)	25
7.10. Tolerância a falhas (degradação graciosa)	26
8. Checklist Finalizada dos Requisitos	27
8.1. Características obrigatórias	27
8.2. Modelos / paradigmas	28
8.3. Cenário — jogo online multijogador	28
8.4. Formato e entregáveis	28
9. Guia de Deploy na AWS (forma automatizada)	30
9.1. Decisões que valem para todo o deploy	30
9.2. Topologia — as 6 máquinas	30
9.3. Pré-requisitos	31
9.4. Security Group (firewall)	31
9.5. Forma automatizada: deploy das 6 a partir de uma máquina	31
9.6. Alternativa manual (um script por EC2)	33
9.7. Bateria de testes na AWS	33
9.8. Desligar para não gastar	34
10. Como Rodar Localmente	35
10.1. Stack completa com Docker (recomendada)	35
10.2. Alternativa sem Docker (stack lite)	36
10.3. Portas da stack local	36
11. Clientes Simulados e Dados de Teste	37
11.1. Clientes simulados (harness de concorrência)	37
11.2. Dados de teste versionados	37
12. Equipe	38

1. Visão Geral do Projeto

O Spaceship é uma plataforma web de três minijogos de naves multiplayer em tempo real, construída como demonstração prática dos conceitos de Software Concorrente e Distribuído (SCD). O sistema reúne, de forma integrada, uma arquitetura de microsserviços orientada a eventos, um servidor autoritativo de jogo, o uso de mais de uma linguagem de programação e os três paradigmas de interação distribuída exigidos pela disciplina: cliente-servidor, publish-subscribe e messaging.

O jogador acessa o sistema pelo navegador, faz login ou registro, escolhe um dos três minijogos e joga em salas (lobby) com outros jogadores. Toda a física, detecção de colisão e contagem de pontuação são processadas no servidor autoritativo: o cliente apenas envia comandos de teclado e renderiza o estado recebido a aproximadamente 60 quadros por segundo. Cada jogador possui 1 ponto de vida (um único acerto é fatal) e cada minijogo tem o seu próprio ranking global.

1.1. Objetivo

Exercitar, de forma integrada, os principais problemas e soluções de sistemas distribuídos e programação concorrente — concorrência sobre dados compartilhados, processamento no servidor concorrente com os clientes, interação remota síncrona e assíncrona, replicação e particionamento, e tratamentos de consistência e disponibilidade — usando tecnologias de relevância atual, em um cenário de jogo online multijogador.

1.2. Os três minijogos

Minijogo	Modo	Jogadores	Inimigos	Objetivo
Jogo 1 — Duelo	PvP competitivo	exatamente 2	nenhum (PvP)	Abrir 2 pontos de vantagem no placar (placar acumulativo).
Jogo 2 — Chuva de Asteroides	Survival cooperativo	1 a 4	asteroides (geração procedural, dificuldade crescente)	Sobreviver e destruir asteroides (1 pt cada).
Jogo 3 — Invasão Alienígena	Cooperativo + boss	1 a 4	Caçador (10 pts), Destruidor (50 pts), boss Leviatã (500 pts)	Sobreviver às ondas e derrotar o boss.

1.3. Características-chave

- **Servidor autoritativo:** toda a lógica de jogo roda no servidor; o cliente é apenas entrada e renderização.
- **Microsserviços orientados a eventos:** cada serviço é especialista, na sua própria linguagem, comunicando-se por múltiplos protocolos.
- **Três linguagens:** C# (Auth), JavaScript/Node.js (Gateway e Engines) e Python (Score).
- **Três paradigmas de comunicação:** cliente-servidor (REST + WebSocket), RPC síncrono (gRPC) e publish-subscribe/messaging (Kafka).
- **Replicação e particionamento:** particionamento funcional (3 engines isolados), particionamento de dados (tabela de pontuações por minijogo) e replicação Primary-Replica do PostgreSQL.

- **Tolerância a falhas:** consistência eventual no ranking via Kafka; degradação graciosa quando um componente cai.

2. A Proposta do Jogo (Game Design)

Esta seção descreve por completo a proposta de jogo: tema, mecânicas, regras, os três minijogos, os inimigos, as telas de interface e a identidade visual.

2.1. Tema e mecânicas base

Tema: combate espacial cooperativo e competitivo em partidas rápidas, com visão 2D de cima (top-down) e orientação vertical. Público-alvo: estudantes e jogadores casuais (12+). Os três minijogos compartilham o mesmo controle fundamental:

- A nave do jogador é representada por um triângulo.
- Movimentação horizontal: teclas ← / → ou A / D.
- Disparo de projétil: tecla Espaço.
- Cada jogador possui 1 ponto de HP — um único acerto é fatal.

Plataforma: navegador web (HTML5 Canvas API). Resolução recomendada: 1280 × 720 px ou superior. Navegadores compatíveis: Chrome 90+, Firefox 88+, Edge 90+.

2.2. Controles e regras universais

Ação	Tecla
Mover para a esquerda	← ou A
Mover para a direita	→ ou D
Atirar projétil	Espaço

- Cada jogador possui 1 HP. Qualquer colisão com projétil inimigo ou obstáculo resulta em morte imediata.
- Não há sistema de respawn. Após morrer, o jogador observa a partida até o fim.
- Os projéteis saem da ponta do triângulo (frente da nave) em linha reta.
- **Movimento autoritativo a 60 Hz:** a velocidade não depende do FPS/máquina do cliente — todo o deslocamento é calculado no servidor.

2.3. Fluxo do jogador

1. O jogador acessa o jogo pelo navegador e realiza login ou registro.
2. Após autenticação, é exibido o menu de seleção com os 3 minijogos disponíveis.
3. O jogador seleciona um minijogo e entra na sala de espera (lobby) correspondente.
4. Ao atingir o número de jogadores necessário, a partida inicia.
5. O cliente envia apenas inputs de teclado; o servidor processa toda a lógica e devolve o estado.
6. Ao término da partida, o resultado é publicado no Kafka e processado pelo Score Service.
7. O jogador retorna ao menu e pode consultar o leaderboard (ranking global por minijogo).

2.4. Minijogo 1 — Duelo (PvP 1v1)

Jogadores: exatamente 2. Objetivo: eliminar o adversário antes de ser eliminado.

- As duas naves aparecem em posições opostas na tela; ambos atiram e se esquivam simultaneamente.
- Ao ser atingido, o round termina e o sobrevivente ganha 1 ponto.

- O placar é acumulativo: o ponto soma ao vencedor do round e o adversário mantém os pontos que já tinha (não há reset).
- A partida termina assim que um jogador fica 2 pontos à frente do outro (menor partida possível: 2×0).

Condição de Vitória: abrir 2 pontos de vantagem no placar. No código, a constante `WIN_MARGIN = 2` controla essa margem. Antes de cada round há uma contagem regressiva de 3 segundos com os inputs travados.

2.5. Minijogo 2 — Chuva de Asteroides (Survival)

Jogadores: 1 a 4 (cooperativo). Objetivo: sobreviver o maior tempo possível destruindo ou desviando dos asteroides que caem.

- Asteroides descem verticalmente; um asteroide que toca uma nave mata o jogador instantaneamente.
- Asteroides podem ser destruídos com projéteis — cada asteroide destruído vale 1 ponto (exibido em tempo real).
- A dificuldade aumenta progressivamente: a frequência e a velocidade dos asteroides crescem conforme a pontuação sobe.
- A partida continua enquanto ao menos um jogador estiver vivo; não há respawn.

Condição de Fim: todos os jogadores morrem. A pontuação da run é compartilhada e registrada para todos os participantes.

2.6. Minijogo 3 — Invasão Alienígena (Cooperativo + Boss)

Jogadores: 1 a 4 (cooperativo). Objetivo: cooperar para sobreviver às ondas de naves inimigas e derrotar o boss final (Leviatã).

- Naves inimigas descem em ondas a partir do topo da tela; os jogadores podem se posicionar livremente.
- Pontuação por destruição: Caçador = 10 pts, Destruidor = 50 pts, Leviatã = 500 pts.
- Após um tempo de combate (`BOSS_TRIGGER_TIME`, ~40 s a 60 Hz) e a eliminação do último inimigo comum, o boss surge.
- A partida continua enquanto ao menos um jogador estiver vivo.

Condição de Vitória: derrotar o boss final. **Condição de Derrota:** todos os jogadores morrem antes de derrotar o boss.

Transição de entrada do chefe (anti-spawn repentino) — fluxo de fases controlado pelo servidor:

8. Combate: ondas de inimigos comuns (Caçador e Destruidor) descem normalmente.
9. Alerta (freeze): atingido o tempo de combate e eliminado o último inimigo comum, o gameplay congela por ~3 segundos (`BOSS_WARNING_TICKS`) e a tela exibe o aviso "A NAVE MÃE SE APROXIMA" com contador regressivo; os projéteis inimigos são limpos para ninguém morrer durante o congelamento.
10. Entrada: o chefe surge no topo e desce em animação até a posição de combate antes de atacar.
11. Batalha: o Leviatã passa a se mover lateralmente e a disparar projéteis com dispersão.

2.7. Inimigos (Minijogo 3)

Inimigo	Forma	HP	Padrão de tiro	Pontos
Caçador	Triângulo invertido	1	1 projétil reto descendente	10
Destruidor	Quadrado	5	3 projéteis (1 reto + 2 diagonais)	50
Leviatã (boss)	Hexágono (com barra de HP)	100	Projéteis descendentes com dispersão horizontal aleatória	500

Nomes internos no código: 'batalha' = Caçador, 'tanque' = Destruidor, 'mae' = Leviatã (chefe).

2.8. Interface do usuário (telas)

- **Login / Registro:** campos de usuário e senha, com confirmação de senha obrigatória no cadastro e feedback de erro.
- **Menu principal:** seleção entre os 3 minijogos, acesso ao leaderboard global e logout.
- **HUD em partida:** indicador de HP; identificação visual dos jogadores; placar de rounds (Jogo 1); contador de vivos (Jogos 2 e 3); indicador de chegada do boss (Jogo 3).
- **Tela de resultado:** vencedor/sobreviventes, pontuação da partida e botão de retorno ao menu.
- **Leaderboard:** ranking global por minijogo (nome, pontuação), atualizado de forma assíncrona pelo Score Service.

2.9. Identidade visual

Estética minimalista sci-fi: fundo preto com elementos geométricos simples; todos os sprites são desenhados via Canvas API, sem arquivos de imagem externos. Cada jogador recebe uma cor única da paleta, atribuída na ordem de entrada na sala, e o projétil herda obrigatoriamente a cor da nave que o disparou (facilita a leitura de quem atirou cada tiro).

Elemento	Cor
Fundo	Preto (#0A0A0F)
Jogador 1 / 2 / 3 / 4	Ciano #00FFFF · Magenta #FF00FF · Verde #00FF88 · Amarelo #FFE600
Inimigo Caçador / Destruidor	Vermelho #FF3333 · Roxo #CC44FF
Boss Leviatã	Vermelho escuro pulsante (#990000)
Projétil do jogador / inimigo	Cor da nave que disparou · Laranja #FF5500
Asteroides	Cinza (#888888)

2.10. Áudio

Efeito de tiro implementado via Web Audio API (sem arquivos externos), disparado apenas quando a nave pessoal do jogador atira, nos três minijogos. Efeitos de explosão e música ambiente por minijogo estão previstos como evolução futura.

3. Stack Tecnológica

A solução combina tecnologias de relevância atual, escolhidas por afinidade com a responsabilidade de cada serviço. O quadro abaixo resume a stack por camada/serviço.

Camada / Serviço	Tecnologia	Papel no sistema
Frontend (cliente web)	HTML5 Canvas API + JavaScript (ES6+)	Login/menu, render da partida e captura de inputs no navegador.
Auth Service	C# / ASP.NET Core, Entity Framework Core, BCrypt, PostgreSQL	Contas, hashing de senha, emissão de JWT e validação de sessão por gRPC.
Game Gateway (relay)	Node.js, Socket.io, socket.io-client	Ponto de entrada do cliente: autenticação, lobby/matchmaking e roteamento (relay).
Game Engines (×3)	Node.js, Socket.io, kafkajs	Loop de física a 60 Hz, colisões e pontuação — um processo isolado por minijogo.
Score Service	Python, Flask, kafka-python, redis-py, pycopg2	Consome o Kafka, materializa o ranking no Redis e o histórico no PostgreSQL; expõe REST.
Mensageria	Apache Kafka (+ ZooKeeper no ambiente local)	Barramento de eventos de fim de partida — desacoplado e persistente.
Dados	PostgreSQL 15 (primário + réplica) · Redis 7	Persistência (usuários e tabela de pontuações particionada) · cache do ranking.
Comunicação	REST/HTTP · WebSocket (Socket.io) · gRPC · Kafka	Os três paradigmas de interação distribuída exigidos.
Infra de testes local	Docker / Docker Compose	Sobe a stack completa em contêineres para desenvolvimento e validação.
Infra de produção	AWS EC2 (Ubuntu) + systemd, sem Docker / sem Kubernetes / sem ALB	Deploy distribuído nativo em 6 instâncias, automatizado por scripts.
Clientes simulados	Node.js (harness de bots)	Bots que autenticam, conectam e jogam os 3 minijogos em paralelo para exercitar a concorrência.

3.1. Por que mais de uma linguagem

- **C# / ASP.NET Core no Auth:** robustez e segurança para contas, hashing (BCrypt) e tokens; multi-threading nativo do Kestrel.
- **Node.js no Gateway e nos Engines:** I/O não bloqueante orientado a eventos, ideal para muitas conexões WebSocket e para o loop de jogo.
- **Python no Score:** flexibilidade e facilidade de manipulação de dados para ranking e persistência.

4. Arquitetura Distribuída

4.1. Estilo arquitetural

A solução adota uma arquitetura baseada em microsserviços e orientada a eventos (EDA), fragmentada em componentes especialistas que usam diferentes modelos de concorrência e se comunicam por múltiplos protocolos. O fluxo de jogo segue o padrão de Servidor Autoritativo: o cliente web apenas envia comandos de entrada e renderiza o estado, enquanto o servidor valida e processa toda a lógica, colisões e pontuação. A topologia em caixas reflete o deploy real na AWS (6 instâncias EC2), sem orquestrador de contêineres (sem Kubernetes) e sem Application Load Balancer — o balanceamento das partidas é feito pelo matchmaking do gateway.

4.2. Serviços e responsabilidades

Serviço	Linguagem / Tecnologia	Responsabilidade	Concorrência
Frontend	HTML5 Canvas + JS	Login, menu, render da partida, captura de inputs	Event-driven no navegador
Auth Service	C# / ASP.NET Core + EF Core + PostgreSQL	Contas, BCrypt, JWT; gRPC de validação de sessão	Multi-threading (Kestrel)
Game Gateway (relay)	Node.js + Socket.io	Autentica, faz lobby/matchmaking e roteia inputs↔estado entre cliente e engine	Event loop não-bloqueante
Game Engines ×3	Node.js (1 processo por jogo)	Loop de física a 60 Hz, colisões, pontuação — isolados por GAME	setInterval por sala
Score Service	Python / Flask	Consume match-completed do Kafka, materializa ranking no Redis e histórico no PostgreSQL; expõe GET /api/scores	Thread de consumo + REST
Message Broker	Apache Kafka	Barramento de eventos de fim de partida (desacoplado, persistente)	—
Dados	PostgreSQL (primário+réplica) · Redis	Persistência (usuários, scores particionada) · cache do ranking	—

4.3. Diagrama da arquitetura

O diagrama abaixo mostra os serviços, o agrupamento por instância EC2 e os fluxos de comunicação numerados (1 a 9). A legenda completa dos fluxos está na subseção 4.5.

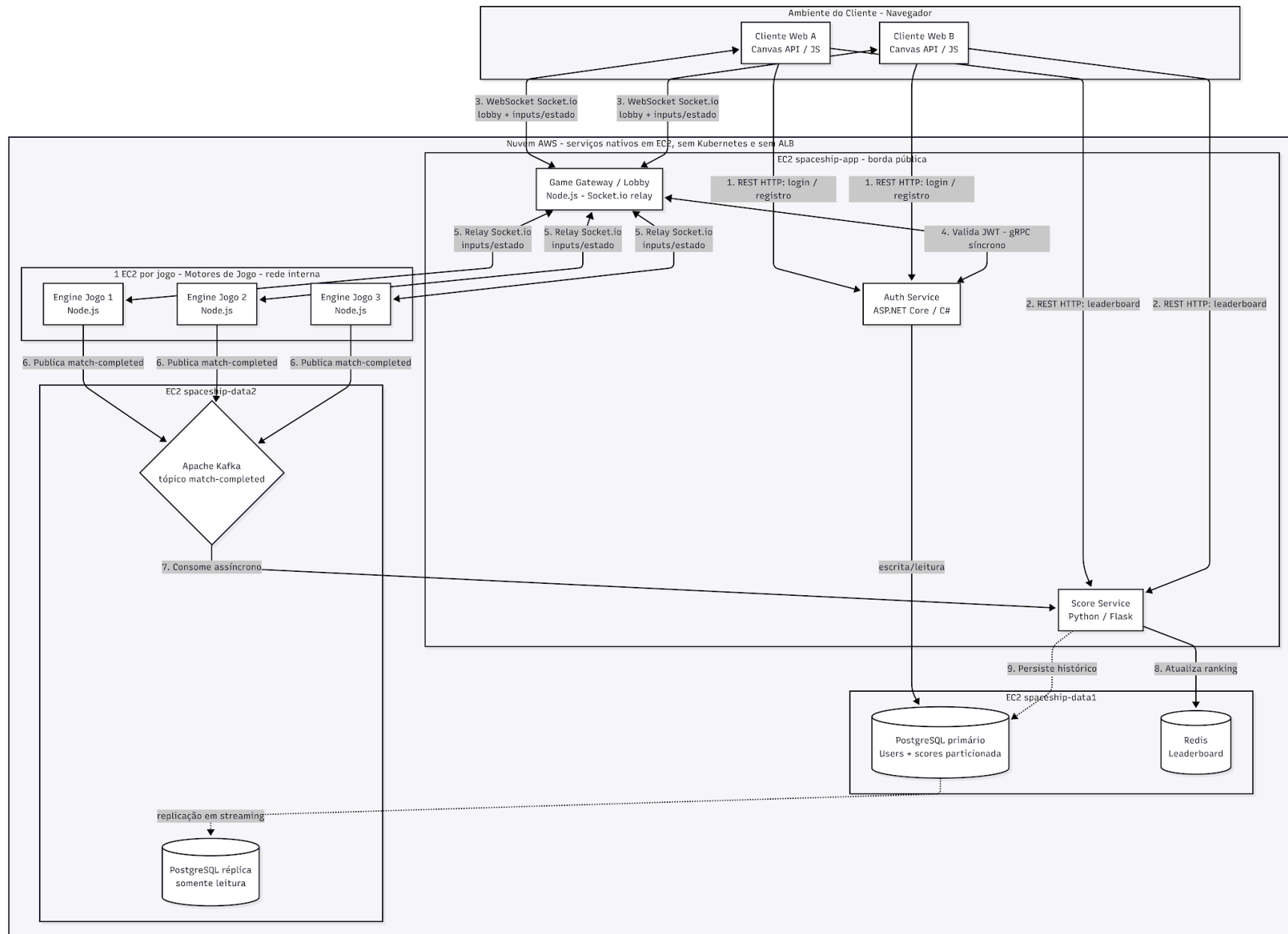


Figura 1 — Arquitetura distribuída do Spaceship: serviços, instâncias EC2 e os três paradigmas de comunicação (cliente-servidor, gRPC síncrono e pub-sub/messaging).

4.4. Paradigmas de comunicação

O sistema demonstra a convivência de três paradigmas distintos de comunicação distribuída:

4.4.1. Cliente-Servidor (REST + WebSocket)

REST/HTTP é usado para ações síncronas de ciclo curto (login, registro e consulta ao leaderboard). WebSocket (Socket.io) é usado para o lobby e para o fluxo contínuo do jogo em tempo real (inputs do cliente e estado por tick). O cliente fala apenas com o gateway; o gateway abre uma conexão-relay a cada engine e repassa os inputs (cliente→engine) e o estado por tick (engine→cliente).

4.4.2. RPC síncrono (gRPC)

Quando um cliente se conecta ao Game Gateway por WebSocket, o gateway valida se o token JWT ainda é válido e ativo chamando o Auth Service por uma RPC gRPC síncrona (bloqueante para aquela conexão). O Auth confere a assinatura/emissor/audiência/expiração do token e também que o usuário ainda existe no banco — ou seja, remover um usuário invalida novas conexões (revogação real). Caso o Auth esteja inacessível (erro de transporte), o gateway cai para validação local do JWT, sem derrubar o jogo.

4.4.3. Publish-Subscribe / Messaging (Kafka)

Ao fim de cada partida, o Game Engine publica um evento match-completed no tópico do Kafka, contendo o minijogo e o resultado. O engine se desvincula imediatamente dessa informação e segue livre para a próxima partida (desacoplamento). O Score Service, inscrito nesse tópico, consome a mensagem no seu próprio ritmo, atualiza o ranking e persiste o histórico. Se o Score cair, o Kafka retém os eventos em disco (consistência eventual).

4.5. Legenda dos fluxos do diagrama

#	Origem → Destino	Protocolo / paradigma	Para quê
1	Navegador → Auth	REST/HTTP (cliente-servidor, síncrono)	Login e registro
2	Navegador → Score	REST/HTTP (cliente-servidor, síncrono)	Ler o ranking global
3	Navegador ↔ Gateway	WebSocket / Socket.io (cliente-servidor)	Lobby e jogo em tempo real (inputs + estado por tick)
4	Gateway ↔ Auth	gRPC síncrono (bloqueante)	Validar a sessão (JWT + revogação) na conexão
5	Gateway ↔ Engines	Socket.io (relay, rede interna)	Repassar inputs ao engine da sala e o estado de volta
6	Engines → Kafka	Publish (assíncrono)	Publicar match-completed ao fim da partida
7	Kafka → Score	Subscribe / messaging (assíncrono)	Consumir o resultado e materializar o ranking
8	Score → Redis	comando	Ranking por minijogo (sorted set)
9	Score → PostgreSQL	persistência	Histórico (scores particionada por minijogo)

4.6. Concorrência em cada serviço

- **Frontend:** event-driven no navegador (eventos de teclado e de rede assíncronos).
- **Auth:** multi-threading do Kestrel/ASP.NET (pool de threads do CLR) para requisições HTTP paralelas.
- **Gateway:** single-threaded event loop não bloqueante, para muitas conexões WebSocket simultâneas.
- **Engines:** um setInterval por sala (60 Hz), isolando o estado de cada partida ativa.
- **Score:** thread de fundo para o consumo do Kafka, separada da API REST que serve o ranking.

4.7. Replicação e particionamento

- **Particionamento funcional:** os 3 minijogos rodam em processos/instâncias separados do Game Engine. Derrubar o engine de um jogo não afeta os outros dois.
- **Particionamento de dados:** a tabela scores é PARTITION BY LIST da coluna game — cada minijogo tem sua partição física (scores_jogo1/2/3 + uma partição DEFAULT), de modo que consultas por minijogo varrem apenas a partição relevante (partition pruning).
- **Replicação de dados:** o PostgreSQL roda em topologia Primary-Replica (streaming replication). O primário recebe as escritas; a réplica, somente leitura, atende leituras concorrentes. Primário e réplica ficam em instâncias EC2 diferentes.

4.8. Consistência e disponibilidade

- **Consistência transacional no Auth:** hashing BCrypt e gravação via EF Core em transações.
- **Consistência eventual no ranking:** o Kafka retém os eventos de fim de partida; o consumidor só confirma o offset depois de persistir (commit manual, semântica at-least-once). Se o Score ou o banco caírem, nenhum resultado é perdido — o processamento retoma do ponto em que parou.
- **Disponibilidade:** réplica de leitura do PostgreSQL; serviços systemd que sobem no boot e reiniciam se caírem; degradação graciosa quando um componente fica indisponível.

5. Funcionamento do Sistema (fluxo de ponta a ponta)

Esta seção descreve o que acontece, em sequência, desde o acesso do jogador até a atualização do ranking global.

5.1. Autenticação e entrada

12. O navegador carrega o frontend e o jogador faz registro/login via REST no Auth Service. O Auth verifica as credenciais (hash BCrypt) e devolve um token JWT assinado (válido por 2 horas).
13. O cliente abre uma conexão WebSocket (Socket.io) com o Gateway, enviando o JWT no handshake.
14. No handshake, o Gateway valida a sessão chamando o Auth por gRPC síncrono: confere a assinatura do token e a existência do usuário. Se o Auth estiver inacessível, o Gateway valida o JWT localmente (fallback) para não derrubar o jogo.

5.2. Lobby e partida em tempo real

15. Autenticado, o cliente entra no lobby do minijogo escolhido. O Gateway mantém, para cada cliente, uma conexão-relay aberta com cada um dos 3 engines.
16. Cada evento do cliente é roteado para o engine dono do jogo (mapa evento→jogo). O cliente envia apenas inputs (movimento/tiro); o engine aplica a física autoritativa a 60 Hz.
17. A cada tick, o engine calcula movimento, projéteis, colisões e pontuação, e emite o estado da sala a cada participante. O Gateway repassa esse estado ao cliente, que apenas renderiza.

5.3. Fim de partida e ranking (pipeline assíncrono)

18. Quando a partida termina, o engine publica um evento match-completed no Kafka, com o minijogo e a lista de jogadores/resultado. O engine não conhece o Score Service.
19. O Score Service consome o evento (thread de fundo), atualiza o ranking no Redis e grava o histórico na tabela particionada do PostgreSQL; só então confirma o offset no Kafka.
20. O frontend lê o ranking por REST no Score Service (GET /api/scores) e exibe o leaderboard.

Formato do evento publicado no Kafka: { game, finishedAt, players: [{ username, score, won }] }

Semântica de ranking por minijogo:

- Jogo 1 (PvP/Duelo): o ranking conta vitórias — apenas o vencedor pontua (ZINCRBY +1).
- Jogos 2 e 3 (cooperativo): o ranking guarda a melhor pontuação (ZADD GT — mantém o máximo).

6. Estrutura de Pastas

Organização do repositório (pastas principais):

Trabalho-Scd-Grupo-6/	
├── frontend/	Cliente web (HTML/CSS/JS, Canvas). Config em config.js.
│ ├── index.html	
│ ├── config.js	Host/portas dos serviços (nada chumbado).
│ └── Dockerfile	
├── auth-service/	Auth canônico em C# / ASP.NET Core.
│ ├── Program.cs	Kestrel (REST 5000 + gRPC 5005), JWT, EF Core.
│ ├── Controllers/	AuthController (register/login/wins).
│ ├── Services/	AuthValidationService (gRPC de validação).
│ ├── Models/	User.
│ ├── Data/	AppDbContext (EF Core).
│ └── Protos/	auth.proto (contrato gRPC).
└── auth-service-lite/	Espelho do Auth em Node.js (users.json) para DEV sem Docker.
├── game-gateway/	Gateway/relay em Node.js (Socket.io).
│ ├── index.js	Autentica, lobby e roteamento (relay).
│ ├── authClient.js	Validação de sessão via gRPC (com fallback local).
│ └── auth.proto	
├── game-engine/	Game Engine em Node.js (a mesma imagem serve os 3 jogos).
│ ├── server.js	Boot, autenticação, escolha do jogo por GAME.
│ ├── kafka.js	Produtor Kafka (publica match-completed).
│ ├── shared/constants.js	Constantes padronizadas (velocidade, cooldown, cores).
│ └── games/	jogo1.js, jogo2.js, jogo3.js (física de cada minijogo).
├── score-service/	Serviço de pontuação em Python/Flask.
│ ├── app.py	API REST (/api/scores, /health) + bootstrap.
│ ├── consumer.py	Consumidor Kafka (thread de fundo, commit manual).
│ └── store.py	Persistência: Redis (ranking) + PostgreSQL (scores
particionada).	
│ ├── config.py	Configuração por variável de ambiente.
│ └── selftest*.py	Testes offline (políticas de ranking e particionamento).
├── load-test/	Clientes simulados (bots) – harness de concorrência.
├── test-data/	Dados de teste versionados (usuários seed + fixtures).
├── deploy/aws/	Deploy nativo na AWS (systemd, sem Docker).
│ ├── deploy-all.sh	Orquestrador (deploy das 6 EC2 a partir de 1 máquina).
│ ├── deploy1..6.sh	Um script por máquina.
│ └── lib/	Pecas por serviço (postgres, redis, kafka, auth,
engine...).	
│ └── common.sh, env.aws.example, maquinas.aws.example	
├── docker-compose.yml	Stack completa de desenvolvimento local.
├── dev-stack.sh	Stack lite (sem Docker) para DEV.
└── .env.example	Modelo de configuração do backend (segredos/portas).

7. Detalhes da Implementação

7.1. Frontend (cliente web)

HTML5 Canvas API + JavaScript nativo. Apresenta login/registro, o menu dos 3 jogos e renderiza a partida em tempo real. O fluxo de jogo usa WebSocket (Socket.io) sempre através do gateway — o cliente não fala direto com os engines. Os endpoints (host/portas de Auth, Gateway e Score) ficam centralizados em `frontend/config.js`: cada campo de host vazio cai para o host atual do navegador, então o mesmo arquivo serve do localhost à AWS sem nada chumbado no `index.html`. A cadência de tiro do cliente espelha a do servidor para evitar ruído sonoro ao segurar a tecla de disparo.

7.2. Auth Service (C# / ASP.NET Core)

Servidor de autenticação canônico. Pontos centrais:

- Registro/login por REST (`/api/auth/register` e `/api/auth/login`). Senhas com hash BCrypt; nunca em texto puro.
- Emissão de JWT assinado (HMAC-SHA256), com claims `sub` (usuário), `jti` e `id`, e validade de 2 horas.
- Persistência via Entity Framework Core sobre PostgreSQL (entidade `User`, com contador de vitórias `Wins` do Jogo 1).
- **gRPC de validação de sessão**: o método `ValidateToken` valida assinatura, emissor, audiência, expiração e confirma que o usuário ainda existe no banco (revogação real).
- **Kestrel com dois protocolos**: REST em HTTP/1.1 na porta 5000 e gRPC em HTTP/2 (h2c) na porta 5005, configurados em `Program.cs`.
- O segredo do JWT vem sempre de variável de ambiente; sem ele, a aplicação não sobe (falha explícita).

7.3. Auth Service Lite (Node.js)

Espelho do Auth em Node.js, com armazenamento em arquivo (`users.json`), para rodar a stack de desenvolvimento sem Docker. Mantém o mesmo contrato e os mesmos claims do Auth canônico (inclusive BCrypt), permitindo testar clientes simulados e fluxo de login sem subir o C#/Postgres.

7.4. Game Gateway (relay)

Ponto de entrada do cliente. Não roda física: autentica, faz lobby/matchmaking e roteia.

- No handshake, valida a sessão via gRPC no Auth (com fallback local em caso de erro de transporte).
- Para cada cliente, abre uma conexão-relay autenticada a cada engine; o `id` do cliente viaja como `playerId`.
- Roteia cada evento do cliente para o engine dono do jogo (mapa evento→jogo) e repassa de volta qualquer evento emitido pelo engine.
- Endereços dos engines e segredo do JWT vêm de variáveis de ambiente (`ENGINE1/2/3_URL`, `JWT_SECRET`, `AUTH_GRPC_URL`).

7.5. Game Engines (3 processos isolados)

A mesma imagem serve os 3 jogos; a variável `GAME` (`jogo1|jogo2|jogo3`) escolhe o módulo de física carregado. Cada engine valida o JWT repassado pelo gateway, mantém o estado das salas e roda o loop de física a 60 Hz. Constantes padronizadas (em `shared/constants.js`) garantem que velocidade de nave, velocidade/raio de projétil, cooldown de tiro e cores sejam idênticos nos 3 jogos e independentes do FPS do cliente.

Parâmetro	Valor padronizado
Tick rate	60 Hz (1000/60 ms)
Velocidade base da nave (<code>SHIP_SPEED</code>)	7 px/tick
Velocidade do projétil (<code>PROJECTILE_SPEED</code>)	14 px/tick
Cadência de tiro (<code>SHOOT_COOLDOWN</code>)	280 ms
Raio do projétil (<code>PROJECTILE_RADIUS</code>)	4 px
Cor do projétil	herdada da nave que disparou

Particularidades por jogo:

- **Jogo 1 (Duelo):** placar acumulativo com vitória por 2 de vantagem (`WIN_MARGIN`); ao fim, incrementa as vitórias do vencedor no Auth (REST) e publica o resultado no Kafka.
- **Jogo 2 (Asteroides):** geração procedural com dificuldade crescente conforme a pontuação; trava de criação de sala por nome único; só o dono da sala inicia a partida.
- **Jogo 3 (Invasão):** máquina de fases (combate → alerta congelado → entrada do boss → batalha); tipos de inimigo com HP e padrões de tiro distintos; encerra liberando a sala (evita jogador-fantasma).

7.6. Score Service (Python / Flask)

Terceira linguagem do sistema. Tem duas frentes que rodam juntas:

- **Consumidor (thread de fundo):** lê o tópico `match-completed` do Kafka (`auto_offset_reset='earliest'`, `group_id` fixo), persiste cada partida e só então confirma o offset (`enable_auto_commit=False`). Em erro de broker/Redis/PostgreSQL, faz backoff exponencial e reconecta sem derrubar a API.
- **API REST:** GET `/api/scores` (por minijogo ou os três de uma vez), GET `/api/scores/player` (recorde do jogador em cada jogo) e GET `/health`. CORS liberado para leitura pelo frontend.
- **Persistência:** Redis (sorted set por minijogo) para leitura rápida do ranking e PostgreSQL (tabela `scores` particionada por minijogo) para o histórico. A política de ranking é uma função pura (`leaderboard_ops`), testável offline pelos selftests.

7.7. Produtor Kafka (resiliência)

No Game Engine, a publicação de `match-completed` é tolerante a falha: a conexão com o broker é preguiçosa e resiliente (reconecta sob demanda), de modo que, se o Kafka subir depois do engine, a próxima partida já publica. Se o broker estiver fora, a publicação vira `no-op` e nunca lança exceção — a partida não quebra. Com o Score parado mas o broker no ar, o Kafka retém as mensagens.

7.8. Persistência: particionamento e replicação

A tabela scores é criada pelo próprio Score Service no primeiro boot:

```
CREATE TABLE scores (  
  id BIGINT GENERATED ALWAYS AS IDENTITY,  
  game VARCHAR(16) NOT NULL,  
  username VARCHAR(64) NOT NULL,  
  score INTEGER NOT NULL DEFAULT 0,  
  won BOOLEAN NOT NULL DEFAULT FALSE,  
  finished_at TIMESTAMPTZ NOT NULL DEFAULT now(),  
  PRIMARY KEY (id, game)  
) PARTITION BY LIST (game);  
-- uma partição física por minijogo + DEFAULT para jogos futuros  
CREATE TABLE scores_jogo1 PARTITION OF scores FOR VALUES IN ('jogo1');  
CREATE TABLE scores_jogo2 PARTITION OF scores FOR VALUES IN ('jogo2');  
CREATE TABLE scores_jogo3 PARTITION OF scores FOR VALUES IN ('jogo3');  
CREATE TABLE scores_outros PARTITION OF scores DEFAULT;
```

Com isso, uma consulta WHERE game='jogo2' varre apenas a partição scores_jogo2 (partition pruning). A replicação do PostgreSQL é configurada pelos scripts de deploy (wal_level=replica, max_wal_senders, papel de replicação e pg_basebackup -R na réplica), deixando o primário para escrita e a réplica somente-leitura para consultas.

7.9. Configuração (nada chumbado)

Nenhum IP ou segredo fica chumbado no código. Há três pontos de configuração, um por cenário:

Arquivo	Quem lê	Para quê
.env (copie de .env.example)	docker-compose.yml (back-end local)	Segredos e portas da stack Docker.
deploy/aws/env.aws	os scripts deploy/aws/*.sh	Segredos + IPs privados/públicos das 6 EC2.
frontend/config.js	o navegador	Host/portas dos serviços que o cliente acessa (Auth, Gateway, Score).

Regra dos endereços na AWS: *_PRIVATE = IP privado da VPC (comunicação entre serviços); *_PUBLIC = DNS público da EC2 (o que o navegador acessa).

7.10. Tolerância a falhas (degradação graciosa)

- Sem o segredo JWT, Auth/Gateway/Engine abortam de propósito (falha explícita, sem default inseguro).
- Auth gRPC fora do ar: o gateway cai para validação local do JWT (o jogo continua).
- Kafka fora: a publicação vira no-op; quando volta, a próxima partida já publica.
- Score/Redis/PostgreSQL fora: o Kafka retém os eventos; a leitura do ranking responde 503 sem derrubar a API.
- Particionamento funcional: derrubar o engine de um jogo não afeta os outros dois.

8. Checklist Finalizada dos Requisitos

Mapeamento dos requisitos da especificação do Trabalho Final ao que foi implementado no projeto.
Legenda: ✓ cumprido.

8.1. Características obrigatórias

#	Requisito	Status	Como é cumprido
1.1	Acessível a múltiplos clientes na Internet	✓	Deploy distribuído na AWS (EC2), com a instância spaceship-app como porta de entrada pública (Frontend, Auth, Gateway, Score).
1.2	Integração e coordenação de componentes distribuídos	✓	8 componentes reais coordenados no fluxo engine→Kafka→Score→Redis/PG→API→tela.
1.3	Acessos concorrentes a dados compartilhados	✓	Estado de sala modificado por vários jogadores simultaneamente; escritas/leituras concorrentes no PostgreSQL.
1.4	Processamento no servidor, concorrente com os clientes	✓	Servidor autoritativo com loop de física a 60 Hz por sala; várias partidas em paralelo.
1.5	Interação remota síncrona e assíncrona	✓	gRPC (síncrono) + Kafka (assíncrono), além de REST (síncrono) e WebSocket (push assíncrono de estado).
1.6	Replicação e particionamento	✓	Particionamento funcional (3 engines) + particionamento de dados (scores por minijogo) + replicação Primary-Replica do PostgreSQL.
1.7	Consistência de dados e disponibilidade	✓	Consistência eventual (Kafka + commit pós-persistência) e réplica de leitura para disponibilidade.

8.2. Modelos / paradigmas

#	Requisito	Status
2.1	Mais de uma linguagem	✓ C# + JavaScript + Python
2.2	Cliente-Servidor	✓ REST + WebSocket
2.3	Publish-Subscribe	✓ Kafka (engines publicam, Score assina)
2.4	Messaging (assíncrono)	✓ Produtor/consumidor match-completed

8.3. Cenário — jogo online multijogador

#	Requisito	Status
3.1	Vários jogadores veem o estado compartilhado	✓ estado por tick a todos da sala
3.2	Jogadores executam ações que mudam o estado	✓ inputs aplicados pelo servidor autoritativo
3.3	Notificações de mudanças de outros jogadores	✓ broadcast de estado/eventos da sala
3.4	Notificações por manutenção das regras do jogo	✓ fases do boss, dificuldade progressiva, geração de inimigos

8.4. Formato e entregáveis

#	Requisito	Status	Observação
4.1	Grupo de 3–4 alunos	✓	4 integrantes (ver seção 12).
4.2	Serviços + clientes simulados	✓	Harness de bots joga os 3 minijogos em paralelo.
4.3	Cenário de demonstração representativo	✓	Roteiro de concorrência/falha no harness + bateria de testes na AWS.
4.4	Demonstração na AWS (EC2)	✓	Deploy distribuído em 6 EC2, automatizado por scripts.
4.5	Código-fonte (e executáveis)	✓	Repositório + imagens Docker por serviço.
4.6	Documentação (arquitetura e implementação)	✓	Este documento.
4.7	Instruções de uso (readme)	✓	Incluídas neste documento (seções 9 e 10).
4.8	Dados de teste	✓	Usuários seed + fixtures de ranking versionados.
4.9	Vídeo de demonstração (todos participam)	✓	Produzido pela equipe.

9. Guia de Deploy na AWS (forma automatizada)

Esta seção mostra como colocar o sistema no ar na AWS pela forma automatizada e mais rápida: uma única máquina orquestradora faz o deploy das 6 instâncias sozinha. O guia assume que você já sabe abrir uma EC2, ver o IP, conectar por SSH e criar um Security Group; o foco é o que instalar, qual script rodar, com qual configuração, em que ordem e como testar.

9.1. Decisões que valem para todo o deploy

- Sem Docker na AWS: cada serviço roda nativo via systemd (sobe no boot e reinicia se cair).
- Sem Kubernetes e sem Application Load Balancer: o balanceamento das partidas é o matchmaking do gateway.
- 6 máquinas (não 10): a conta AWS permite 9 instâncias; serviços que não competem por porta foram agrupados, mantendo separados o que mais importa — o PostgreSQL primário × réplica e os 3 engines isolados.
- AMI: Ubuntu Server 22.04 / 24.04 LTS. Tipos: t3.medium nas 3 máquinas mais carregadas (data1, data2, app) e t3.small em cada engine.
- O navegador fala só com a spaceship-app; os engines ficam na rede interna da VPC (sem porta pública).

9.2. Topologia — as 6 máquinas

#	Nome (tag Name)	Script	Serviços	Portas	Tipo
1	spaceship-data1	deploy1.sh	PostgreSQL primário + Redis	5432 · 6379 (interno)	t3.medium
2	spaceship-data2	deploy2.sh	Kafka + PostgreSQL réplica	9092 (+9093) · 5432 (interno)	t3.medium
3	spaceship-app	deploy3.sh	Auth + Gateway + Score + Frontend	80, 3000, 5000, 8000 (públicas) + 5005 (interno)	t3.medium
4	spaceship-engine1	deploy4.sh	Game Engine jogo1	4001 (interno)	t3.small
5	spaceship-engine2	deploy5.sh	Game Engine jogo2	4002 (interno)	t3.small
6	spaceship-engine3	deploy6.sh	Game Engine jogo3	4003 (interno)	t3.small

Acesso interno = só as outras máquinas alcançam, pelo IP privado da VPC. Acesso público = porta aberta no Security Group. Apenas a spaceship-app tem portas públicas.

9.3. Pré-requisitos

21. Um par de chaves SSH (arquivo .pem) criado.
22. Uma VPC padrão para as 6 EC2 (mesma VPC/região, para se enxergarem pelo IP privado). Anote o CIDR da sub-rede (na VPC padrão, 172.31.0.0/16).
23. As 6 instâncias EC2 (Ubuntu) criadas, cada uma com a tag Name correspondente da tabela acima.

9.4. Security Group (firewall)

Toda porta nasce fechada para a internet; só abre quando você cria uma regra de entrada (inbound). Use um único Security Group (spaceship-sg) para as 6 instâncias, com 6 regras de entrada: 1 interna + 5 públicas.

Regra	Type (menu)	Protocolo	Port range	Source	Para quê
1	All traffic	All	All	o próprio spaceship-sg	Malha interna (banco, Redis, Kafka, gRPC, relay dos engines)
2	SSH	TCP	22	0.0.0.0/0	Administrar via SSH
3	HTTP	TCP	80	0.0.0.0/0	Abrir o site (spaceship-app)
4	Custom TCP	TCP	3000	0.0.0.0/0	Jogo em tempo real (gateway / Socket.io)
5	Custom TCP	TCP	5000	0.0.0.0/0	Login e cadastro (Auth REST)
6	Custom TCP	TCP	8000	0.0.0.0/0	Ler o ranking global (Score)

A Regra 1 (origem = o próprio SG) faz banco, Redis, Kafka, gRPC do Auth e o relay gateway↔engines funcionarem sem expor nada à internet. As portas 5432, 6379, 9092/9093, 5005 e 4001–4003 nunca recebem regra pública — ficam vivas só dentro da VPC.

9.5. Forma automatizada: deploy das 6 a partir de uma máquina

Em vez de repetir os passos em cada EC2, suba uma 7ª EC2 (a orquestradora) e rode um comando: ela conecta nas 6 por SSH, descobre os endereços, gera a configuração, envia o código e roda os scripts na ordem certa.

9.5.1. Preparar a 7ª máquina (uma vez)

24. Abra mais uma EC2 Ubuntu (uma t3.small basta — ela só orquestra). Coloque-a no mesmo Security Group spaceship-sg: assim a Regra 1 já libera o SSH dela para as 6 pelo IP privado, sem abrir porta nova.

25. Instale o necessário, clone o repositório e traga a chave .pem das EC2:

```
sudo apt-get update -y && sudo apt-get install -y git rsync openssh-client openssl
git clone <URL_DO_REPO> && cd Trabalho-Scd-Grupo-6
# copie a sua chave .pem para esta máquina (nome esperado: spaceship-key.pem)
chmod 600 spaceship-key.pem
```

9.5.2. Rodar o deploy

```
cp deploy/aws/maquinas.aws.example deploy/aws/maquinas.aws
nano deploy/aws/maquinas.aws      # preencha SO o *_SSH de cada maquina e o SSH_KEY
bash deploy/aws/deploy-all.sh
```

No arquivo maquinas.aws você quase não preenche nada — só como conectar em cada EC2 (campos *_SSH) e o caminho da chave (SSH_KEY). O IP privado e o DNS público de cada máquina o script

descobre sozinho (perguntando à própria EC2 via metadata), então isso sobrevive à troca do DNS público quando você desliga/religa as instâncias — basta rodar de novo. Os segredos (JWT_SECRET e senhas) são gerados na primeira vez e reaproveitados nas próximas (ficam no env.aws, que não vai para o Git).

Como preencher *_SSH:

- 7ª máquina dentro do spaceship-sg (recomendado): use o IP privado de cada EC2.
- 7ª máquina fora (seu PC): use o DNS público de cada EC2 e libere a porta 22 vinda do seu IP no Security Group.

9.5.3. O que o orquestrador faz (na ordem)

data1 → data2 → app em série (a réplica do Postgres depende do primário; o app depende de Postgres/Redis/Kafka) e os 3 engines em paralelo (são isolados — ganho de tempo). Ao final, faz um smoke test e imprime o link do jogo. Logs por máquina ficam em deploy/aws/.deploy-logs/. É idempotente: pode rodar quantas vezes quiser.

9.5.4. Opções úteis

Comando	Para quê
<code>bash deploy/aws/deploy-all.sh --dry-run</code>	Só mostra o plano e os endereços descobertos; não altera nada.
<code>bash deploy/aws/deploy-all.sh --yes</code>	Não pergunta "continuar?" (bom para automatizar).
<code>bash deploy/aws/deploy-all.sh --roles=engine2</code>	Re-deploya só uma máquina (as outras já no ar).
<code>bash deploy/aws/deploy-all.sh --skip-sync</code>	Não reenvia o código; só regenera o env.aws e roda os deploys.

9.6. Alternativa manual (um script por EC2)

A forma automatizada usa, por baixo, exatamente os mesmos scripts. Se preferir o manual, em cada EC2 clone o repositório, copie deploy/aws/env.aws.example para env.aws, preencha segredos e IPs/DNS das 6 máquinas e rode o script da máquina, nesta ordem:

```
sudo bash deploy/aws/deploy1.sh # spaceship-data1 - Postgres primario + Redis [PRIMEIRO]
sudo bash deploy/aws/deploy2.sh # spaceship-data2 - Kafka + Postgres replica
sudo bash deploy/aws/deploy3.sh # spaceship-app - Auth + Gateway + Score + Frontend
sudo bash deploy/aws/deploy4.sh # spaceship-engine1 (jogo1)
sudo bash deploy/aws/deploy5.sh # spaceship-engine2 (jogo2)
sudo bash deploy/aws/deploy6.sh # spaceship-engine3 (jogo3)
```

Estado/logs: `sudo systemctl status spaceship-<svc>` · `sudo journalctl -u spaceship-<svc> -f` (Postgres e Redis usam os nomes nativos postgresql e redis-server).

9.7. Bateria de testes na AWS

Abra `http://<DNS_PÚBLICO_DA_spaceship-app>`, registre, faça login e jogue os 3 minijogos. Abaixo, <APP_PUB> é esse DNS público.

```
# Smoke test
curl -s -o /dev/null -w "frontend %{http_code}\n" http://<APP_PUB>
curl -s -o /dev/null -w "auth      %{http_code}\n"
http://<APP_PUB>:5000/api/auth/wins/x    # 404 = no ar
curl -s -o /dev/null -w "gateway  %{http_code}\n" http://<APP_PUB>:3000/socket.io/
# 400 = no ar
curl -s "http://<APP_PUB>:8000/health"

# Concorrencia / clientes simulados
cd load-test && npm install
node run.js --bots=8 --game=mix --gateway=http://<APP_PUB>:3000
--auth=http://<APP_PUB>:5000/api/auth
```

- **gRPC (revogação):** na spaceship-app, `journalctl -u spaceship-gateway | grep -i grpc`; remova um usuário no Postgres primário e tente reconectar — a conexão é recusada.
- **Particionamento:** na spaceship-data1, `EXPLAIN (COSTS OFF) SELECT * FROM scores WHERE game='jogo2'`; deve citar só `scores_jogo2`.
- **Replicação:** na spaceship-data2 (réplica), `SELECT pg_is_in_recovery()`; deve dar `t`; escreva no primário (jogue/rode os bots) e leia a tabela `scores` na réplica — as linhas aparecem propagadas.
- **Tolerância a falha / isolamento:** na spaceship-engine1, `sudo systemctl stop spaceship-engine-jogo1` — Jogos 2 e 3 seguem normais; só o Jogo 1 fica indisponível.

9.8. Desligar para não gastar

Pare (sem terminar) as 6 instâncias quando não estiver testando — os serviços `systemd` sobem sozinhos ao religar a EC2. Religue a `spaceship-data1` antes da `spaceship-data2` (a réplica espera o primário).

10. Como Rodar Localmente

Para desenvolvimento e validação local, há duas formas: a stack completa em Docker (recomendada, sobe tudo, inclusive Kafka/Redis/PostgreSQL) e uma stack lite sem Docker.

10.1. Stack completa com Docker (recomendada)

Pré-requisitos: Docker e Docker Compose. Rode todos os comandos na raiz do projeto.

```
cp .env.example .env          # (1a vez) defina POSTGRES_PASSWORD e JWT_SECRET
sudo docker compose up -d --build
```

A primeira execução demora (build das imagens). Ao terminar, abra <http://localhost:8080>. Sem `POSTGRES_PASSWORD` ou `JWT_SECRET` no `.env`, o `docker compose up` aborta de propósito (com mensagem clara).

10.1.1. Verificar que subiu

```
sudo docker compose ps
curl -s -o /dev/null -w "frontend :8080 -> %{http_code}\n" http://localhost:8080
curl -s -o /dev/null -w "auth :5000 -> %{http_code}\n"
http://localhost:5000/api/auth/wins/healthcheck
curl -s -o /dev/null -w "score :8000 -> %{http_code}\n"
http://localhost:8000/health
```

10.1.2. Popular o ranking com bots (opcional)

```
cd load-test && npm install && node run.js --bots=8 --game=mix; cd ..
curl -s "http://localhost:8000/api/scores?limit=5" | python3 -m json.tool
```

10.1.3. Verificar os mecanismos distribuídos

gRPC (validação de sessão):

```
sudo docker compose logs game-gateway | grep -i grpc
# revogacao: remova um usuario e tente reconectar -- o gateway recusa
sudo docker exec -it spaceship_postgres psql -U postgres -d authdb \
  -c "DELETE FROM \"Users\" WHERE \"Username\"='SEU_USER';"
```

Pipeline assíncrono (fim de partida → Kafka → Score → Redis):

```
sudo docker compose logs game-engine-2 | grep -i kafka
sudo docker exec spaceship_redis redis-cli ZREVRANGE leaderboard:jogo2 0 -1
WITHSCORES
sudo docker compose logs --tail=30 score-service
```

Particionamento de dados por minijogo:

```
sudo docker exec -it spaceship_postgres psql -U postgres -d authdb -c "\d+ scores"
sudo docker exec -it spaceship_postgres psql -U postgres -d authdb \
  -c "EXPLAIN (COSTS OFF) SELECT * FROM scores WHERE game='jogo2';"
# esperado: o EXPLAIN cita so scores_jogo2 (partition pruning)
```

10.1.4. Derrubar tudo

```
sudo docker compose down          # mantém os dados do banco (volume)
# sudo docker compose down -v     # também APAGA os dados (zera Postgres e ranking)
```

10.2. Alternativa sem Docker (stack lite)

Para testar os clientes simulados e o auth sem Kafka/Redis/PostgreSQL (o ranking global fica indisponível, degradando sem quebrar). Usa o auth-service-lite:

```
./dev-stack.sh deps      # (1a vez) instala node_modules dos servicos
./dev-stack.sh start     # sobe auth(5000) + engines(4001/2/3) + gateway(3000)
python3 -m http.server 8080 --directory frontend # para jogar no navegador
./dev-stack.sh stop      # derruba e libera as portas
```

10.3. Portas da stack local

Serviço	Porta	Observação
Frontend (nginx)	8080	abrir no navegador
Auth (C#)	5000 / 5005	REST /api/auth · gRPC (validação)
Gateway (relay)	3000	Socket.io (cliente↔gateway)
Game Engines 1/2/3	4001/4002/4003	Socket.io interno (relay do gateway)
Score Service	8000	GET /api/scores
PostgreSQL	5432	tabela scores particionada
Redis	6379	ranking (sorted sets)
Kafka / ZooKeeper	9092 / 2181	tópico match-completed

11. Clientes Simulados e Dados de Teste

11.1. Clientes simulados (harness de concorrência)

O diretório load-test contém bots que simulam jogadores reais: autenticam no Auth (REST, como o navegador), conectam no Gateway por Socket.io (passando pelo relay e pelos 3 engines) e jogam os 3 minijogos automaticamente, em paralelo. Ao final, imprimem um relatório de carga (conexões, partidas iniciadas/concluídas, pico de partidas simultâneas, throughput de inputs e estados, latências e erros categorizados). Atende ao requisito de clientes simulados e dá base ao cenário de demonstração.

```
cd load-test && npm install
node run.js # 8 bots, mix dos 3 jogos (default)
node run.js --bots=16 --game=mix
node run.js --bots=4 --game=jogo2
node run.js --gateway=http://SEU_HOST:3000 --auth=http://SEU_HOST:5000/api/auth
```

Demonstração de tolerância a falha (particionamento funcional): com a stack no ar, derrube só o engine do Jogo 1 (por exemplo, fuser -k 4001/tcp). O Jogo 3 continua concluindo partidas (isolado), enquanto o Jogo 1 degrada com erro de timeout, sem crash.

11.2. Dados de teste versionados

Arquivo	Para que serve	Quem consome
seed-users.json	Usuários de teste (login/senha) para os clientes simulados	harness de bots e seed.js
sample-match-events.json	Fixture determinístico de eventos match-completed (payload do Kafka)	selftest do Score
expected-rankings.json	Oráculo: ranking esperado após aplicar o fixture	selftest do Score

São 12 pilotos com a mesma senha padrão; o harness registra de forma idempotente (ignora "já existe") e autentica cada um antes de jogar. O fixture de ranking valida, offline (sem Kafka/Redis/PostgreSQL), as duas políticas de ranking do Score: vitórias no Jogo 1 (ZINCRBY) e melhor pontuação nos Jogos 2 e 3 (ZADD GT). A validação roda por:

```
cd score-service && python selftest_dataset.py # saída esperada: SELFTEST DATASET OK
```

12. Equipe

Integrante	Função
Moisés Ferreira Protázio Bastos	Game Design
Thiago Nascente Borges	Programação
Vinicius Silva Benevides	Programação
Ana Liz Bomfim Gomes	Testes / QA

Trabalho Final da disciplina de Software Concorrente e Distribuído — Instituto de Informática, Universidade Federal de Goiás, 2026.1.